

10.9 I/O 重定向

Linux shell 提供了 I/O 重定向操作符，允许用户将磁盘文件和标准输入输出联系起来。例如，键入

```
linux> ls > foo.txt
```

使得 shell 加载和执行 ls 程序，将标准输出重定向到磁盘文件 foo.txt。就如我们将会在 11.5 节中看到的那样，当一个 Web 服务器代表客户端运行 CGI 程序时，它就执行一种相似类型的重定向。那么 I/O 重定向是如何工作的呢？一种方式是使用 dup2 函数。

```
#include <unistd.h>

int dup2(int oldfd, int newfd);
```

返回：若成功则为非负的描述符，若出错则为 -1。

dup2 函数复制描述符表表项 oldfd 到描述符表表项 newfd，覆盖描述符表表项 newfd 以前的内容。如果 newfd 已经打开了，dup2 会在复制 oldfd 之前关闭 newfd。

假设在调用 dup2(4,1)之前，我们的状态如图 10-12 所示，其中描述符 1(标准输出)对应于文件 A(比如一个终端)，描述符 4 对应于文件 B(比如一个磁盘文件)。A 和 B 的引用计数都等于 1。图 10-15 显示了调用 dup2(4,1)之后的情况。两个描述符现在都指向文件 B；文件 A 已经被关闭了，并且它的文件表和 v-node 表表项也已经被删除了；文件 B 的引用计数已经增加了。从此以后，任何写到标准输出的数据都被重定向到文件 B。

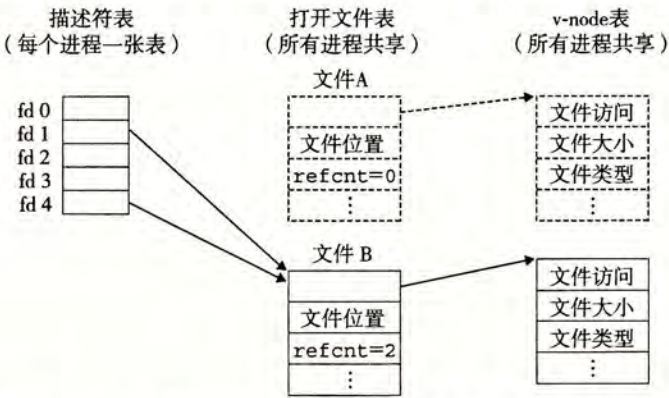


图 10-15 通过调用 dup2(4,1)重定向标准输出之后的内核数据结构。初始状态如图 10-12 所示

旁注 左边和右边的 hoinkies

为了避免和其他括号类型操作符比如 “]” 和 “[” 相混淆，我们总是将 shell 的 “>” 操作符称为 “右 hoinky”，而将 “<” 操作符称为 “左 hoinky”。

- 练习题 10.4 如何用 dup2 将标准输入重定向到描述符 5?
- 练习题 10.5 假设磁盘文件 foobar.txt 由 6 个 ASCII 码字符 “foobar” 组成，那么下列程序的输出是什么?

```
1 #include "csapp.h"
2
3 int main()
```

```

4  {
5      int fd1, fd2;
6      char c;
7
8      fd1 = Open("foobar.txt", O_RDONLY, 0);
9      fd2 = Open("foobar.txt", O_RDONLY, 0);
10     Read(fd2, &c, 1);
11     Dup2(fd2, fd1);
12     Read(fd1, &c, 1);
13     printf("c = %c\n", c);
14     exit(0);
15 }

```

10.10 标准 I/O

C 语言定义了一组高级输入输出函数，称为标准 I/O 库，为程序员提供了 Unix I/O 的较高级别的替代。这个库(libc)提供了打开和关闭文件的函数(fopen 和 fclose)、读和写字节的函数(fread 和 fwrite)、读和写字符串的函数(fgets 和 fputs)，以及复杂的格式化的 I/O 函数(scanf 和 printf)。

标准 I/O 库将一个打开的文件模型化为一个流。对于程序员而言，一个流就是一个指向 FILE 类型的结构的指针。每个 ANSI C 程序开始时都有三个打开的流 stdin、stdout 和 stderr，分别对应于标准输入、标准输出和标准错误：

```

#include <stdio.h>
extern FILE *stdin; /* Standard input (descriptor 0) */
extern FILE *stdout; /* Standard output (descriptor 1) */
extern FILE *stderr; /* Standard error (descriptor 2) */

```

类型为 FILE 的流是对文件描述符和流缓冲区的抽象。流缓冲区的目的和 RIO 读缓冲区的一样：就是使开销较高的 Linux I/O 系统调用的数量尽可能得小。例如，假设我们有一个程序，它反复调用标准 I/O 的 getc 函数，每次调用返回文件的下一个字符。当第一次调用 getc 时，库通过调用一次 read 函数来填充流缓冲区，然后将缓冲区中的第一个字节返回给应用程序。只要缓冲区中还有未读的字节，接下来对 getc 的调用就能直接从流缓冲区得到服务。

10.11 综合：我该使用哪些 I/O 函数？

图 10-16 总结了我们在这一章里讨论过的各种 I/O 包。

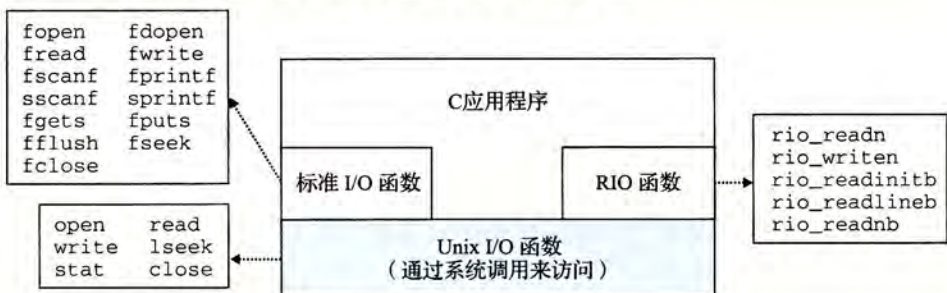


图 10-16 Unix I/O、标准 I/O 和 RIO 之间的关系